MACHINE LEARNING MODELS IN DEPTH

From concepts and mathematics to implementation, code and model evaluation

A hands-on Python and scikit-learn guide

Includes regression • classification • trees • ensembles • clustering • dimensionality reduction
SVMs • probabilistic models • neural networks • time-series forecasting

How to use this guide

This guide is designed as a study reference, not a catalogue of algorithms to memorise. For each model family, focus on five questions: What problem does it solve? What assumptions does it make? How does it learn? When is it a good choice? How do we know whether it has learned something useful?

The examples use Python, pandas, NumPy, matplotlib and scikit-learn. Most snippets are small enough to run in a notebook. Install the core packages with:

```
python -m pip install numpy pandas matplotlib seaborn scikit-learn
```

For neural-network examples, install PyTorch separately. Time-series sections use statsmodels:

```
python -m pip install torch statsmodels
```

A learning sequence

Start with the shared ML workflow and evaluation principles.
Study linear and logistic regression; they establish the language of model fitting, loss and regularisation.
Move to decision trees, random forests and boosting; these are powerful on tabular data.
Study distance- and margin-based models such as k-nearest neighbours and support vector machines.
Learn unsupervised learning: clustering and dimensionality reduction.
Then study neural networks and time-series forecasting.
Finally, build a complete project with a clean baseline, validation strategy, error analysis and reproducible pipeline.

1. The machine-learning problem

1.1 What is a machine-learning model?

A machine-learning model is a mathematical function or learned procedure that maps input information to an output. During training, an algorithm uses examples to adjust the model's parameters so that its predictions or representations become useful for a defined objective.

For supervised learning, a dataset contains inputs X and known targets y. A model learns an approximation $\hat{f}$ such that $\hat{f}(X)$ predicts y for observations it has not seen. In unsupervised learning, targets are absent; the model searches for structure such as groups, lower-dimensional representations or unusual observations.

1.2 Main learning settings

Setting	What the data contains	Typical tasks	Examples
Supervised	Inputs X and target y	Regression, classification	Predict crop price; classify a transaction

Unsupervised	Inputs X, no labelled target	Clustering, representation learning, anomaly detection	Group farms by production profile
Semi-supervised	A small labelled set plus many unlabelled examples	Classification with limited labels	Classify crop disease when expert labels are scarce
Self-supervised	Labels are generated from the data itself	Pretraining representations	Learn image or text representations
Reinforcement learning	States, actions, rewards and transitions	Sequential decision-making	Control irrigation decisions under a reward objective

1.3 Regression versus classification

Regression predicts a numerical quantity: price, yield, rainfall or demand. Classification predicts a category: disease/no disease, low/medium/high risk, or whether a buyer will accept an offer. A probability output from a classifier is not automatically a calibrated probability; calibration must be assessed separately.

1.4 Features, targets, parameters and hyperparameters

Feature: an input variable used by the model, such as crop, market, date, grade or rainfall.
Target: the value the model is trained to predict, such as price per kilogram.
Parameter: a value learned from data, such as a regression coefficient or a tree split.
Hyperparameter: a design choice set before or during model selection, such as tree depth, regularisation strength or number of neighbours.
Prediction: the output produced for a new observation.

1.5 The generalisation goal

A model can fit its training data extremely well and still perform poorly on new data. The goal is not to memorise the training set; it is to generalise. Underfitting occurs when a model is too simple or poorly trained to capture important patterns. Overfitting occurs when it captures noise or accidental patterns that do not persist.

Situation	Training performance	Validation/test performance	Likely interpretation
Underfitting	Poor	Poor	Model too simple, weak features, or inadequate training
Healthy fit	Good	Similar and good	Useful generalisation
Overfitting	Excellent	Much worse	Model has learned noise or leakage

2. The end-to-end ML workflow

2.1 Define the prediction task before choosing a model

Write down the unit of prediction, target, prediction horizon, intended user and cost of errors. For example: 'Using information available by the end of day t, predict the median wholesale price per kilogram for maize in market m on day t+1.' This statement prevents vague targets and accidental use of future information.

2.2 Inspect and understand the data

```
import pandas as pd


df = pd.read_csv("observations.csv")
print(df.shape)
print(df.head())
print(df.info())
print(df.isna().sum().sort_values(ascending=False).head(20))
print(df.describe(include="all").T)
```

Check duplicates, impossible values, units, inconsistent category names, date ranges, missingness patterns, target distribution and whether rows represent independent observations. Plot the target and important features. Investigate outliers rather than deleting them automatically.

2.3 Split the data correctly

A random split is appropriate only when rows are plausibly independent and future observations are not being predicted from past observations. Grouped data may require group-aware splitting. Time-ordered data usually requires chronological splitting.

```
from sklearn.model_selection import train_test_split


X = df.drop(columns=["target"])
y = df["target"]


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

Keep the test set untouched until the end. Use training data for fitting and cross-validation; use validation folds or a dedicated validation set for model selection. Repeatedly tuning against the test set turns it into another validation set.

2.4 Preprocess without leakage

Imputation, scaling, encoding, feature selection and dimensionality reduction must be fitted on training data only. A scikit-learn Pipeline makes this safer because each cross-validation fold fits its own transformations.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline

numeric_features = ["rainfall", "temperature", "transport_cost"]
categorical_features = ["crop", "market", "grade"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features)
])
```

2.5 Establish a baseline

A baseline is a deliberately simple reference. For regression, predict the training-set mean or median. For classification, predict the majority class or use a domain rule. A complex model is only useful if it improves on a credible baseline under the same validation strategy.

2.6 Train, validate, compare and inspect

Fit candidate models using training data only.
Evaluate with metrics aligned to the real decision.
Use cross-validation or time-aware backtesting.
Tune a small number of meaningful hyperparameters.
Inspect residuals, confusion matrices, subgroup performance and failure cases.
Choose the simplest model that meets the performance and operational requirements.

3. Linear regression

3.1 Definition and intuition

Linear regression predicts a continuous target as a weighted sum of input features. It is one of the most important baseline models because it is fast, interpretable and exposes the relationship between features, parameters and loss.

For p features, the model is:

$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$

β_0 is the intercept and β_j is the expected change in the prediction for a one-unit increase in x_j, holding the other included features fixed. This is a conditional association, not automatically a causal effect.

3.2 How it learns

Ordinary least squares chooses coefficients that minimise the sum of squared residuals: $\Sigma_i(y_i - \hat{y}_i)^2$. A residual is the observed value minus the prediction. Squaring penalises large errors more heavily and produces a differentiable objective.

3.3 Assumptions and limitations

The conditional mean is adequately represented by a linear combination of the features.
Observations and errors are independent for the usual inferential calculations; time-series and grouped data often violate this.
Severe multicollinearity makes individual coefficients unstable.
Homoscedasticity and approximately well-behaved residuals matter for classical standard errors and inference, though prediction can still be useful when these assumptions are imperfect.
Linear regression is sensitive to influential outliers and does not automatically discover nonlinear relationships or interactions.

3.4 Implementation

```
from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score



model = Pipeline([
```

```
    ("preprocess", preprocess),
    ("regressor", LinearRegression())
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R2:", r2_score(y_test, pred))
```

MAE is the average absolute error and is easy to interpret in the target's units. RMSE penalises large errors more strongly. R^2 measures improvement over predicting the test-set mean; it can be negative on test data and does not tell you whether errors are operationally acceptable.

3.5 When to use it

Use linear regression as a baseline, when interpretability is important, when relationships are approximately additive and linear, or when data is limited. Improve it with meaningful transformations, interaction terms, splines or regularisation when justified.

4. Regularised linear models: Ridge, Lasso and Elastic Net

Regularisation adds a penalty to the training objective to discourage overly large coefficients. It can improve generalisation when features are numerous, correlated or noisy.

4.1 Ridge regression (L2)

Ridge minimises squared error plus $\alpha\Sigma\beta_j^2$. It shrinks coefficients towards zero but generally does not make them exactly zero. It is useful when many features each contribute a small amount and predictors are correlated.

```
from sklearn.linear_model import Ridge

ridge = Pipeline([
    ("preprocess", preprocess),
    ("regressor", Ridge(alpha=1.0))
])
ridge.fit(X_train, y_train)
```

```
ridge_pred = ridge.predict(X_test)
```

4.2 Lasso regression (L1)

Lasso adds $\alpha\Sigma|\beta_j|$ to the loss. Its penalty can force some coefficients to exactly zero, creating a sparse model. It can act as a form of feature selection, but with highly correlated predictors it may select one somewhat arbitrarily.

```
from sklearn.linear_model import Lasso


lasso = Pipeline([
    ("preprocess", preprocess),
    ("regressor", Lasso(alpha=0.05, max_iter=20000))
])
lasso.fit(X_train, y_train)
```

4.3 Elastic Net

Elastic Net combines L1 and L2 penalties. It is often useful when there are correlated groups of features and some sparsity is desirable.

```
from sklearn.linear_model import ElasticNet


elastic = Pipeline([
    ("preprocess", preprocess),
    ("regressor", ElasticNet(alpha=0.05, l1_ratio=0.5, max_iter=20000))
])
elastic.fit(X_train, y_train)
```

The penalty strength α is a hyperparameter. Larger values imply stronger shrinkage. Scaling numeric features is important because penalties operate on coefficient magnitudes. Tune α and, for Elastic Net, l1_ratio using cross-validation.

5. Logistic regression

5.1 Definition

Logistic regression is a supervised classification model. Despite its name, it predicts class probabilities rather than a continuous target. For binary classification it models the log-odds of class 1 as a linear function of the features.

$p(y=1|x) = 1 / (1 + e^{-z})$, where $z = \beta_0 + \beta^T x$. The sigmoid function maps any real number to a value between 0 and 1.

5.2 Decision threshold

A default threshold of 0.5 assigns class 1 when its predicted probability is at least 0.5. The threshold is a decision choice, not a law. If missing a disease case is much more costly than a false alarm, a lower threshold may be appropriate. Select thresholds using validation data and the costs of errors.

5.3 Loss function

Logistic regression commonly minimises binary cross-entropy (log loss). It strongly penalises confident predictions that are wrong. Regularisation is commonly used to control coefficient size.

5.4 Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score


clf = Pipeline([
   ("preprocess", preprocess),
   ("classifier", LogisticRegression(max_iter=2000, class_weight="balanced"))
])
clf.fit(X_train, y_train)


labels = clf.predict(X_test)
probabilities = clf.predict_proba(X_test)[:, 1]


print(confusion_matrix(y_test, labels))
print(classification_report(y_test, labels))
print("ROC AUC:", roc_auc_score(y_test, probabilities))
```

For imbalanced data, accuracy can be misleading. Examine precision, recall, F1, the confusion matrix, PR-AUC and the costs of false positives and false negatives. ROC-AUC measures ranking quality across thresholds; it does not guarantee calibrated probabilities.

5.5 Multiclass classification

Logistic regression can handle multiple classes using multinomial logistic regression or one-vs-rest strategies. Scikit-learn selects the appropriate formulation based on solver and configuration. Always inspect per-class metrics, especially when class frequencies differ.

6. k-Nearest Neighbours (k-NN)

k-NN is a non-parametric, instance-based method. It stores training examples and predicts for a new point by finding the k closest training examples. For regression it averages their targets; for classification it uses a majority vote or distance-weighted vote.

6.1 Distance and the role of k

A common distance is Euclidean distance: $d(x,z)=\sqrt{\Sigma_j (x_j - z_j)^2}$. Other metrics, such as Manhattan or Minkowski distance, may suit different data. Small k produces flexible, noisy boundaries; large k smooths the boundary but may underfit. Feature scaling is essential when units differ.

```
from sklearn.neighbors import KNeighborsClassifier


knn = Pipeline([
   ("preprocess", preprocess),
   ("classifier", KNeighborsClassifier(n_neighbors=7, weights="distance"))
])
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

Strengths: simple, intuitive and capable of nonlinear boundaries. Weaknesses: prediction can be slow on large datasets, irrelevant features damage distance calculations, and performance degrades in high-dimensional spaces (the curse of dimensionality).

7. Decision trees

7.1 Definition and intuition

A decision tree recursively splits the feature space into regions. Each internal node asks a question such as 'Is rainfall less than 50 mm?' Each branch represents an answer, and each leaf produces a prediction. Trees can model nonlinear relationships and interactions without requiring feature scaling.

7.2 How a split is chosen

For regression, a split may be selected to reduce squared error or another impurity measure. For classification, common criteria include Gini impurity and entropy. A split is useful when its child nodes are more homogeneous than the parent.

Gini impurity = $1 - \sum_k p_k^2$. Entropy = $-\sum_k p_k \log_2(p_k)$. Both are zero when a node contains only one class.

7.3 Overfitting and pruning

A fully grown tree can memorise training data. Control complexity with max_depth, min_samples_split, min_samples_leaf, max_leaf_nodes or cost-complexity pruning. A larger minimum leaf size generally creates smoother predictions.

```
from sklearn.tree import DecisionTreeRegressor


tree = Pipeline([
    ("preprocess", preprocess),
    ("regressor", DecisionTreeRegressor(
        max_depth=6, min_samples_leaf=10, random_state=42
    ))
])
tree.fit(X_train, y_train)
tree_pred = tree.predict(X_test)
```

Trees are easy to explain locally, but a single tree can be unstable: small changes in training data may produce a different structure. They also extrapolate poorly for regression because predictions are averages of training targets in leaves.

8. Random forests

A random forest is an ensemble of decision trees. It trains many trees on bootstrap samples of the training data and considers a random subset of features at each split. Regression predictions are averaged; classification predictions are aggregated by voting or class probabilities.

8.1 Why it works

Averaging diverse trees reduces variance. Random feature selection decorrelates trees so that their errors are less likely to move together. Random forests are strong general-purpose models for tabular data and require relatively little feature scaling.

8.2 Important hyperparameters

n_estimators: number of trees; more trees usually stabilise predictions but cost more computation.
max_depth: maximum tree depth.
min_samples_leaf: minimum observations in a leaf; a key smoothness control.
max_features: number or fraction of features considered at each split.
bootstrap: whether each tree is trained on a bootstrap sample.

```
from sklearn.ensemble import RandomForestRegressor


forest = Pipeline([
   ("preprocess", preprocess),
   ("regressor", RandomForestRegressor(
      n_estimators=400,
      max_depth=None,
      min_samples_leaf=3,
      max_features=0.8,
      random_state=42,
      n_jobs=-1
   ))
])
forest.fit(X_train, y_train)
forest_pred = forest.predict(X_test)
```

Random forests can be less interpretable than a single tree and may be large. Built-in impurity feature importance can favour high-cardinality features; permutation importance on held-out data is often a better diagnostic. Correlated features can split importance between themselves.

9. Gradient boosting: AdaBoost, Gradient Boosting, XGBoost-style methods

9.1 Core idea

Boosting builds an ensemble sequentially. Each new learner is trained to improve the current ensemble, often by fitting residuals or the negative gradient of a loss function. Unlike random forests, which mainly reduce variance through parallel averaging, boosting often reduces bias by correcting earlier mistakes.

9.2 Gradient boosting

A gradient-boosted model adds a small tree at each iteration. The learning rate controls how much each new tree contributes. Small trees and a modest learning rate can create a strong model, but too many iterations or overly complex trees can overfit.

```
from sklearn.ensemble import HistGradientBoostingRegressor


hgb = Pipeline([
   ("preprocess", preprocess),
   ("regressor", HistGradientBoostingRegressor(
      learning_rate=0.06,
      max_iter=300,
      max_leaf_nodes=20,
      l2_regularization=1.0,
      random_state=42
   ))
])
hgb.fit(X_train, y_train)
hgb_pred = hgb.predict(X_test)
```

9.3 Modern gradient-boosting libraries

XGBoost, LightGBM and CatBoost are popular implementations with optimisations and additional controls. XGBoost adds regularisation and efficient split finding; LightGBM uses histogram-based learning and leaf-wise growth; CatBoost has specialised handling for categorical features and ordered boosting. Their APIs differ, so consult the relevant library documentation before implementation.

For structured/tabular data, gradient boosting is often a strong contender. Compare it against a random forest and a simple linear baseline rather than assuming it must win.

9.4 Key tuning dimensions

Learning rate and number of boosting iterations are linked: smaller learning rates often need more iterations.
Tree depth or number of leaves controls interaction complexity.
Subsampling rows or columns can reduce overfitting.
Regularisation and minimum leaf constraints limit overly specific rules.
Early stopping uses validation performance to stop when additional iterations no longer help.

10. Support Vector Machines (SVM)

A support vector machine seeks a decision boundary with a large margin between classes. The observations closest to the boundary are support vectors. A larger margin can improve generalisation.

10.1 Linear and kernel SVM

A linear SVM uses a linear boundary. The kernel trick allows an SVM to create nonlinear boundaries by computing similarities as if data had been mapped into a higher-dimensional feature space. Common kernels include linear, polynomial and radial basis function (RBF).

10.2 C and gamma

C controls the penalty for margin violations. Large C tries harder to classify training examples correctly and can overfit; small C allows a wider margin with more violations.

For an RBF kernel, gamma controls how local each training point's influence is. Large gamma can create intricate boundaries; small gamma produces smoother boundaries.

```
from sklearn.svm import SVC


svm = Pipeline([
    ("preprocess", preprocess),
    ("classifier", SVC(
        kernel="rbf",
        C=2.0,
        gamma="scale",
        probability=True,
        class_weight="balanced",
        random_state=42
    ))
])
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)
```

SVMs are sensitive to feature scale. They can work well on medium-sized, high-dimensional datasets, but kernel SVM training can become expensive as the number of observations grows. Probability estimation adds extra computation.

11. Naive Bayes

Naive Bayes is a family of probabilistic classifiers based on Bayes' theorem. It estimates P(class|features) from P(features|class) and P(class). The 'naive' assumption is that features are conditionally independent given the class. This assumption is often unrealistic, but the model can still work surprisingly well.

Common variants: Gaussian Naive Bayes for continuous features assumed to follow class-conditional Gaussian distributions; Multinomial Naive Bayes for counts such as word frequencies; Bernoulli Naive Bayes for binary features.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline


# Example assumes all features are numeric.
nb = Pipeline([
   ("imputer", SimpleImputer(strategy="median")),
   ("scaler", StandardScaler()),
   ("classifier", GaussianNB())
])
nb.fit(X_train[numeric_features], y_train)
```

Naive Bayes is fast and useful as a baseline, especially for text classification. Its probability estimates can be poorly calibrated when independence assumptions are violated. Choose the variant to match the feature representation.

12. Clustering: learning without labels

12.1 What clustering does

Clustering groups observations based on similarity. There is no target label telling the algorithm what the correct groups are. The result depends on the features, distance measure, scaling, algorithm and chosen hyperparameters. A cluster is a mathematical grouping, not automatically a meaningful real-world segment.

12.2 K-means

K-means partitions observations into K clusters by assigning each point to its nearest centroid and updating each centroid to the mean of its assigned points. It minimises within-cluster sum of squared distances.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline


cluster_pipe = Pipeline([
   ("imputer", SimpleImputer(strategy="median")),
   ("scaler", StandardScaler()),
   ("kmeans", KMeans(n_clusters=4, n_init=20, random_state=42))
])
cluster_labels = cluster_pipe.fit_predict(X[numeric_features])
```

K-means is efficient and works best when clusters are roughly compact and similarly sized. It is sensitive to scaling, outliers and initialisation; it requires K to be chosen. Use the elbow plot and silhouette score as diagnostics, not as automatic truth.

12.3 DBSCAN

DBSCAN groups points in dense regions and marks isolated points as noise. It can discover irregularly shaped clusters and does not require specifying the number of clusters. Its main parameters are eps (neighbourhood radius) and min_samples (density requirement). It is sensitive to feature scaling and struggles when cluster densities vary substantially.

```
from sklearn.cluster import DBSCAN


dbscan = Pipeline([
   ("imputer", SimpleImputer(strategy="median")),
   ("scaler", StandardScaler()),
   ("cluster", DBSCAN(eps=0.7, min_samples=8))
])
labels = dbscan.fit_predict(X[numeric_features])
```

12.4 Hierarchical clustering

Agglomerative hierarchical clustering starts with individual observations and repeatedly merges the closest clusters. The resulting dendrogram shows nested groupings. Linkage choices (ward, complete, average, single) affect the shape of the clusters. It can be useful for exploratory segmentation, but naïve implementations can be expensive for large datasets.

12.5 Evaluating clusters

The silhouette coefficient compares how close a point is to its own cluster against the nearest other cluster. Values near 1 indicate well-separated assignments; values around 0 indicate overlapping clusters; negative values suggest possible misassignment. Internal metrics do not establish business usefulness. Profile the clusters and test whether they are stable and actionable.

13. Dimensionality reduction: PCA and related methods

13.1 Principal Component Analysis (PCA)

PCA transforms correlated numeric features into a smaller set of uncorrelated components. The first component captures the greatest possible variance; each later component captures the greatest remaining variance subject to orthogonality. PCA is unsupervised: it does not know the target.

PCA is useful for compression, visualisation, denoising and reducing multicollinearity. It can make interpretation harder because each component is a combination of original features. High variance is not necessarily high predictive value.

```
from sklearn.decomposition import PCA


pca_pipe = Pipeline([
   ("imputer", SimpleImputer(strategy="median")),
   ("scaler", StandardScaler()),
   ("pca", PCA(n_components=0.95))
])
X_reduced = pca_pipe.fit_transform(X[numeric_features])
```

n_components=0.95 asks PCA to retain enough components to explain 95% of training variance. In predictive workflows, fit PCA inside the cross-validation pipeline to avoid leakage.

13.2 t-SNE and UMAP

t-SNE and UMAP are nonlinear methods often used to visualise high-dimensional data in two or three dimensions. They can reveal local structure, but distances between far-apart visual clusters should not be overinterpreted. They are primarily exploratory tools, not default replacements for PCA or predictive features.

14. Neural networks and deep learning

14.1 The basic neuron

A neuron computes a weighted sum $z=w^{T}x+b$ and applies an activation function $a=g(z)$. A neural network layers many such computations. Hidden layers learn intermediate representations; the output layer is shaped for the task.

14.2 Common activation functions

ReLU: max(0,z); common in hidden layers.
Sigmoid: maps to (0,1); often used for a binary output probability.
Softmax: converts multiclass logits into probabilities that sum to one.
Tanh: maps to (−1,1); sometimes useful in recurrent architectures.

14.3 Training: forward pass, loss and backpropagation

A forward pass computes predictions. A loss function measures prediction error. Backpropagation applies the chain rule to compute gradients of the loss with respect to parameters. An optimiser such as stochastic gradient descent or Adam updates parameters. Training repeats this over batches and epochs.

14.4 A small PyTorch regression model

This example assumes X_train and y_train have already been transformed into numeric arrays and scaled appropriately. Neural networks generally require numeric tensors and careful preprocessing.

```
import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

Xtr = torch.tensor(X_train_array, dtype=torch.float32)
ytr = torch.tensor(y_train_array, dtype=torch.float32).view(-1, 1)

loader = DataLoader(
    TensorDataset(Xtr, ytr),
    batch_size=32,
    shuffle=True
)

model = nn.Sequential(
    nn.Linear(Xtr.shape[1], 64),
```

```
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)

loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(100):
    model.train()
    epoch_loss = 0.0
    for xb, yb in loader:
        optimizer.zero_grad()
        predictions = model(xb)
        loss = loss_fn(predictions, yb)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(xb)
    print(epoch + 1, epoch_loss / len(loader.dataset))
```

14.5 Preventing overfitting

Use a validation set and early stopping.
Use dropout or weight decay where appropriate.
Reduce network size when the dataset is small.
Apply data augmentation for images/audio when it preserves the label.
Use batch normalisation when it fits the architecture and training setup.
Compare against simpler models; deep learning is not automatically superior on small tabular datasets.

14.6 Important neural-network families

Family	Core strength	Typical use
MLP / feed-forward	Learns nonlinear feature combinations	Tabular or vector inputs
CNN	Learns local spatial patterns using convolution	Images, spatial grids, some time-series
RNN / LSTM / GRU	Processes sequences with recurrent state	Sequential signals and legacy sequence tasks
Transformer	Uses attention to model relationships across tokens	Text, multimodal data, long sequences

	or time steps	
Autoencoder	Learns a compressed representation by reconstructing inputs	Compression, representation learning, anomaly detection

15. Time-series forecasting

15.1 What makes time series different?

Time-series observations have an order. Randomly shuffling them can leak future information into training. A forecast must be evaluated by training on the past and predicting a later period. Time-series data may contain trend, seasonality, cycles, structural breaks and autocorrelation.

15.2 Naive baselines

Always compare sophisticated forecasts against simple baselines: last observation, seasonal naive (same period last season), moving average or a drift forecast. A model that cannot beat a relevant naive forecast may not add value.

15.3 ARIMA

ARIMA(p,d,q) combines autoregression (p), differencing (d) and a moving-average error component (q). Seasonal ARIMA adds seasonal terms. ARIMA is useful for univariate series with structured temporal dependence, but external drivers require extensions such as ARIMAX/SARIMAX.

```
from statsmodels.tsa.arima.model import ARIMA


# y_train_series must be a chronologically ordered pandas Series.
arima = ARIMA(y_train_series, order=(2, 1, 2))
arima_fit = arima.fit()
forecast = arima_fit.forecast(steps=len(y_test_series))
```

15.4 Supervised learning with lag features

Tree ensembles and linear models can forecast time series after the series is converted into supervised rows. At time t, features may include lagged prices, rolling statistics and calendar variables. Every feature must be available at the forecast origin.

```
def make_lag_features(series, lags=(1, 7, 14, 28)):
    frame = pd.DataFrame({"target": series})
    for lag in lags:
```

```
    frame[f"lag_{lag}"] = frame["target"].shift(lag)
  frame["rolling_mean_7"] = frame["target"].shift(1).rolling(7).mean()
  frame["rolling_std_7"] = frame["target"].shift(1).rolling(7).std()
  return frame.dropna()


supervised = make_lag_features(price_series)
X_ts = supervised.drop(columns="target")
y_ts = supervised["target"]
```

The shift(1) before rolling is important: it ensures the rolling statistic does not include the target value being predicted. For multiple markets or commodities, construct lags within each series group, not across group boundaries.

15.5 Walk-forward validation

A simple backtest trains on an initial historical window, predicts the next period, then advances the forecast origin. Depending on the use case, the model may be retrained at every step or at fixed intervals. Report metrics over multiple forecast origins and horizons.

```
from sklearn.model_selection import TimeSeriesSplit
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np


tscv = TimeSeriesSplit(n_splits=5)
scores = []


for train_idx, valid_idx in tscv.split(X_ts):
  Xtr, Xv = X_ts.iloc[train_idx], X_ts.iloc[valid_idx]
  ytr, yv = y_ts.iloc[train_idx], y_ts.iloc[valid_idx]


  model = RandomForestRegressor(
    n_estimators=300, min_samples_leaf=3, random_state=42
  )
  model.fit(Xtr, ytr)
  pv = model.predict(Xv)


  scores.append({
```

```
    "MAE": mean_absolute_error(yv, pv),
    "RMSE": mean_squared_error(yv, pv) ** 0.5
  })

print(scores)
```

15.6 Forecasting metrics

MAE: interpretable average absolute error.
RMSE: places more weight on large misses.
MAPE: percentage error, but unstable or undefined when actual values are zero or near zero.
sMAPE: a scaled percentage measure, though it has its own interpretation issues.
MASE: scales errors against a naive in-sample forecast and is useful for comparing series.

Always report performance by horizon, commodity, market and relevant season. An average score can hide a model that performs poorly for a particular market or during price spikes.

16. Anomaly detection

Anomaly detection identifies observations that differ substantially from expected patterns. It may be supervised (known anomaly labels), semi-supervised (mostly normal examples) or unsupervised. Common methods include Isolation Forest, Local Outlier Factor, one-class SVM and robust statistical rules.

```
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

anomaly_model = make_pipeline(
  StandardScaler(),
  IsolationForest(contamination=0.03, random_state=42)
)
# Returns 1 for inliers and -1 for detected outliers.
outlier_flags = anomaly_model.fit_predict(X[numeric_features])
```

An anomaly score is not proof of fraud, error or danger. Investigate flagged records, estimate false-alarm rates and include domain review. The contamination parameter is a modelling assumption about the expected fraction of anomalies, not a fact learned automatically.

17. Model evaluation in depth

17.1 Regression metrics

Metric	Meaning	Use / caution
MAE	Mean absolute error	Easy to interpret in target units; robust relative to RMSE
MSE	Mean squared error	Penalises large errors strongly; units are squared
RMSE	Square root of MSE	Same units as target; sensitive to large misses
R^2	Relative fit against a mean baseline	Can be negative on held-out data; not a measure of absolute error
MAPE	Mean absolute percentage error	Problematic near zero and asymmetric in some settings

17.2 Classification metrics

Metric	Definition / interpretation	Question it answers
Accuracy	Correct predictions / all predictions	How often is the model correct overall?
Precision	TP / (TP + FP)	When it predicts positive, how often is it right?
Recall	TP / (TP + FN)	How many actual positives does it find?
Specificity	TN / (TN + FP)	How many actual negatives does it correctly reject?
F1	Harmonic mean of precision and recall	How balanced are precision and recall?
ROC-AUC	Area under ROC curve	How well does it rank positives above negatives?
PR-AUC	Area under precision-recall curve	How useful is ranking when positives are rare?
Log loss	Penalises incorrect probability estimates	Are predicted probabilities useful, not just labels?

17.3 Cross-validation

K-fold cross-validation divides training data into K folds. Each fold is held out once while the model trains on the others. The average provides a more stable estimate than one split. Use stratified folds for many classification tasks, group folds when observations share entities, and time-aware splits for forecasting.

17.4 Calibration

A classifier is calibrated when, among cases assigned probability 0.8, approximately 80% are positive. Reliability diagrams and Brier score can assess calibration. Calibration matters when decisions depend on probability values, expected costs or risk thresholds.

17.5 Error analysis

Do not stop at a single score. Plot regression residuals against predictions, time and key features. For classification, inspect false positives and false negatives. Segment performance by market, crop, season, region, farm size or other operationally meaningful groups. Look for systematic error, missing coverage and drift.

18. Hyperparameter tuning

Hyperparameter tuning searches for configurations that perform well under a validation strategy. Grid search tests a defined grid; random search samples combinations and is often more efficient; Bayesian optimisation uses prior trials to choose promising next trials.

```
from sklearn.model_selection import RandomizedSearchCV, KFold
from sklearn.ensemble import RandomForestRegressor


pipe = Pipeline([
   ("preprocess", preprocess),
   ("model", RandomForestRegressor(random_state=42, n_jobs=-1))
])


param_distributions = {
   "model__n_estimators": [200, 400, 600],
   "model__max_depth": [None, 8, 12, 18],
   "model__min_samples_leaf": [1, 2, 4, 8],
   "model__max_features": [0.5, 0.8, 1.0]
}


search = RandomizedSearchCV(
   pipe,
   param_distributions=param_distributions,
   n_iter=25,
   scoring="neg_mean_absolute_error",
   cv=KFold(n_splits=5, shuffle=True, random_state=42),
```

```
    n_jobs=-1,
    random_state=42
)
search.fit(X_train, y_train)
print(search.best_params_)
print(-search.best_score_)
```

For time-series data, replace ordinary KFold with a chronological strategy such as TimeSeriesSplit or a custom walk-forward backtest. Do not tune on the final test set.

19. Interpretability and explainability

Linear coefficients show the model's learned conditional relationships, but depend on scaling, coding and correlated features.
A shallow decision tree can provide explicit decision rules.
Permutation importance measures how much validation performance worsens when one feature is shuffled.
Partial dependence plots show average model response as a feature changes, but can be misleading when features are strongly correlated.
SHAP-style additive explanations can describe feature contributions for individual predictions; explanations are not causal effects.

Interpretability should be matched to the audience and decision. A model explanation is not proof that the model is correct, fair or causally valid.

20. Common mistakes and how to avoid them

Mistake	Why it is dangerous	Better practice
Preprocessing before splitting	Leaks information from validation/test data	Fit transformations inside a Pipeline
Random split for forecasting	Future patterns leak into training	Chronological split and walk-forward validation
Using accuracy on imbalanced classes	A majority-class model can look strong	Inspect precision, recall, PR-AUC and costs
Tuning on the test set	Makes test performance optimistic	Keep a final untouched test set
Ignoring a baseline	Complexity may add no value	Compare against naive and simple models
Assuming correlation is causation	Model associations can be confounded	Use causal design for causal questions
Deleting all outliers	May remove real high-impact events	Investigate provenance and domain meaning
Ignoring drift	Relationships change over time	Monitor data and performance after deployment
Reporting only one aggregate metric	Hides subgroup failures	Break down errors by important segments

21. A practical project: agricultural market price prediction

A market-price predictor is a natural setting for comparing several model families. The key challenge is not merely selecting an algorithm; it is defining a forecast that is operationally meaningful and building a dataset that respects time.

21.1 Define the target precisely

Example target: predict the next day's median wholesale price per kilogram for a specific crop in a specific market. Alternative targets include next-week average price, a 7-day price range, or the probability that price will rise by more than a chosen threshold. Pick one target and horizon first.

21.2 Candidate features

Feature group	Examples	Important caution
Identity	Crop, variety, grade, market	Use consistent category definitions
Time	Date, day of week, month, season	Derive from forecast date; avoid future leakage
Price history	Lag 1, 7, 14, 28; rolling means	Compute within crop-market series and shift correctly
Supply	Market arrivals, harvest estimates, stock	Use only values available at forecast time
Weather	Rainfall, temperature, forecast anomalies	Distinguish observed from forecast weather
Logistics	Fuel/transport cost, road disruptions	Timestamp when each value became available
Demand	Buyer activity, volume traded, orders	Beware of post-outcome variables

21.3 Models worth comparing

Model	Role in the experiment	What it can reveal
Seasonal naive	Required baseline	How much can simple repetition explain?
Linear / Ridge	Interpretable baseline	Are relationships approximately additive?
Random Forest	Nonlinear tabular benchmark	Are interactions and nonlinearities useful?
Gradient boosting	Strong tabular candidate	Can sequential tree corrections improve accuracy?
SARIMAX / ARIMA	Time-series benchmark	How much temporal structure

		exists in the price series?
Neural network	Later-stage experiment	Is there enough data to justify extra complexity?

21.4 Suggested experiment design

Create a canonical observation table with one row per crop-market-date and a clearly defined price statistic and unit.
Sort by date and identify missing days, duplicate records, changes in measurement practice and extreme values.
Build a seasonal-naive forecast and record its MAE and RMSE.
Create lag and rolling features using only historical values available at each forecast origin.
Use expanding-window or rolling-window backtesting; reserve the latest period as a final test window.
Compare Ridge, Random Forest and gradient boosting using identical forecast origins and metrics.
Report errors by crop, market, season and forecast horizon; investigate price spikes and regime changes.
Save the preprocessing pipeline, model version, feature definitions and training-data cutoff together.

21.5 A compact baseline implementation

```
# Assumes a single crop-market series indexed by date.
# price_series: pandas Series sorted by date, with price as its value.

frame = pd.DataFrame({"price": price_series}).sort_index()
frame["lag_1"] = frame["price"].shift(1)
frame["lag_7"] = frame["price"].shift(7)
frame["lag_14"] = frame["price"].shift(14)
frame["mean_7"] = frame["price"].shift(1).rolling(7).mean()
frame["mean_28"] = frame["price"].shift(1).rolling(28).mean()
frame["month"] = frame.index.month
frame["day_of_week"] = frame.index.dayofweek
frame = frame.dropna()

X = frame.drop(columns="price")
y = frame["price"]

cutoff = int(len(frame) * 0.80)
X_train, X_test = X.iloc[:cutoff], X.iloc[cutoff:]
y_train, y_test = y.iloc[:cutoff], y.iloc[cutoff:]

from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error
```

```
model = HistGradientBoostingRegressor(
    learning_rate=0.05,
    max_iter=300,
    max_leaf_nodes=15,
    l2_regularization=1.0,
    random_state=42
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, pred))
```

This compact example is for one regularly indexed series. Real deployments with multiple crops and markets need group-aware lag construction, missing-date handling, market/crop identifiers, availability timestamps and a proper walk-forward evaluation.

22. Model deployment and monitoring

22.1 Save the full pipeline

```
import joblib


joblib.dump(model, "price_model.joblib")
loaded_model = joblib.load("price_model.joblib")
new_prediction = loaded_model.predict(new_data)
```

Save preprocessing and model together, preferably as one pipeline. Record package versions, training date range, data source, feature schema, target definition and evaluation results. Only load trusted model files because pickle/joblib formats can execute code during deserialisation.

22.2 What to monitor

Input data quality: missing values, invalid ranges, unseen categories and schema changes.
Data drift: whether the feature distribution has changed.
Target drift: whether the target distribution has changed.
Performance drift: whether actual outcomes show worsening error once labels arrive.
Operational reliability: latency, failed predictions, stale data and service availability.
Subgroup performance: whether errors become concentrated in certain markets, crops or seasons.

A model is not finished when it is deployed. Establish retraining criteria, rollback procedures, human review for high-impact cases and a record of model versions.

23. Choosing the right model

Data / problem characteristics	Models to try first	Why
Small-to-medium tabular regression	Linear/Ridge, Random Forest, Gradient Boosting	Strong baselines with manageable complexity
Tabular classification	Logistic Regression, Random Forest, Gradient Boosting	Good balance of performance and practicality
High-dimensional sparse text	Naive Bayes, Logistic Regression, linear SVM	Efficient for sparse feature spaces
Similarity-based small dataset	k-NN, SVM	Useful when local geometry or margins matter
Unlabelled segmentation	K-means, hierarchical, DBSCAN	Different cluster assumptions suit different structures
Many correlated numeric features	Ridge, PCA + model	Shrinkage or compact representation
Large image/audio/text corpus	CNNs, Transformers and other deep models	Can learn rich representations from raw signals
Single ordered series	Naive/seasonal naive, ARIMA, lag-feature boosting	Respect temporal order and benchmark simple forecasts

There is no universally best model. Model choice depends on the target, sample size, feature types, data quality, validation design, latency, interpretability, maintenance and the cost of errors.

24. Glossary

Term	Meaning
Algorithm	Procedure used to fit or construct a model
Model	Learned mapping or representation produced by training
Training	Process of learning model parameters from data
Inference	Using a trained model to produce outputs
Loss function	Quantity optimised during training
Gradient	Derivative that indicates how a parameter affects loss
Epoch	One pass through the training dataset
Batch	Subset of training examples processed together
Regularisation	Constraint or penalty used to control model complexity
Feature engineering	Constructing useful input variables from raw data
Leakage	Information unavailable at prediction time enters training or evaluation
Drift	Change in data or relationships over time

Ensemble	Combination of multiple models
Calibration	Agreement between predicted probabilities and observed frequencies

25. Study exercises

Fit LinearRegression, Ridge and RandomForestRegressor to the same regression dataset. Compare MAE, RMSE and residual plots.

Create a binary classification task. Compare LogisticRegression, KNeighborsClassifier, DecisionTreeClassifier and RandomForestClassifier. Explain the precision-recall trade-off.

Train a deep decision tree and a constrained tree. Compare training and validation scores to demonstrate overfitting.

Use K-means on a scaled dataset. Try several K values, calculate silhouette scores and profile the resulting groups.

Apply PCA to numeric features. Plot explained variance and compare a model trained with and without PCA.

Build a time-series baseline and a lag-feature model. Compare them using chronological backtesting, never a random split.

For the agricultural price prediction task, define the forecast horizon, create a leakage checklist and write down which features are genuinely available at forecast time.

26. Suggested references

Use these as authoritative companions while implementing the material. API details evolve, so check the current documentation when working with a specific version.

Scikit-learn User Guide — https://scikit-learn.org/stable/user_guide.html
Scikit-learn API Reference — https://scikit-learn.org/stable/modules/classes.html
PyTorch Tutorials — https://pytorch.org/tutorials/
Statsmodels Time Series Analysis — https://www.statsmodels.org/stable/tsa.html
Forecasting: Principles and Practice (online textbook) — https://otexts.com/fpp3/

Final advice

Learn models by implementing them against the same well-defined problem. Begin with a baseline, preserve a valid test strategy, inspect errors and explain why one model performs better than another. Understanding the data and evaluation design is often more important than choosing an advanced algorithm.

A useful next milestone is to build a model-comparison notebook for one dataset: baseline → linear model → tree → random forest → boosting → error analysis. Once that workflow is reliable, move to a time-series experiment for agricultural prices.